Sorting Performance Analyzer Report

Name: **Bhavya Shany**

Roll No: **2501730024**

Course: **Data Structures (ETCCDS202)**

Unit: 3 - **Sorting Algorithms**

Submitted To- **Mrs. Neetu Chauhan**

1. Introduction

This report presents the implementation and performance analysis of three sorting algorithms: Insertion Sort, Merge Sort, and Quick Sort. The aim is to compare their practical performance with theoretical time complexities under different input conditions.

2. Experimental Results

Input	Insertion Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
Random 1000	23.945	2.344	2.79
Sorted 1000	0.12	1.529	1.741
Reverse 1000	44.408	3.983	1.689
Random 5000	689.027	30.518	15.887
Sorted 5000	1.015	14.686	18.872
Reverse 5000	1630.536	15.628	17.556
Random 10000	2714.587	32.757	20.386
Sorted 10000	1.158	20.379	20.122
Reverse 10000	4888.048	23.471	18.887

3. Analysis

For random inputs, Merge Sort and Quick Sort perform significantly better than Insertion Sort. This aligns with their O(n log n) time complexity compared to $O(n^2)$ for Insertion Sort.

For sorted data, Insertion Sort performs extremely well because it requires minimal comparisons and shifts. Quick Sort shows slower performance due to poor pivot selection when the last element is used.

For reverse-sorted data, Insertion Sort performs the worst as expected. Merge Sort remains stable, while Quick Sort performs reasonably but may degrade depending on pivot choice.

4. Theory vs Practical Observation

The results clearly validate theoretical expectations. Insertion Sort becomes inefficient as input size increases, while Merge Sort and Quick Sort scale efficiently. Quick Sort shows slight variation due to pivot selection.

Divide & Conquer: Binary Search and Merge Sort both follow this paradigm. In

this experiment, Merge Sort consistently outperformed Insertion Sort on large

datasets because it reduces the problem size by half at each step.

Practical Observation: Insertion Sort outperformed all others on Sorted Data,

demonstrating its O(n) best-case efficiency. However, it showed quadratic growth

(O(n2)) on random and reverse data, becoming extremely slow as size increased.

5. Stability and Memory

- Insertion Sort is stable and in-place.
- Merge Sort is stable but uses extra memory.
- Quick Sort is in-place but not stable in most implementations.

6. Conclusion

The experiment demonstrates that algorithm selection depends on data size and input type. Merge Sort is consistent, Quick Sort is fast on average, and Insertion Sort is suitable for small or nearly sorted data.

Made by:

Bhavya Shany

2501730024